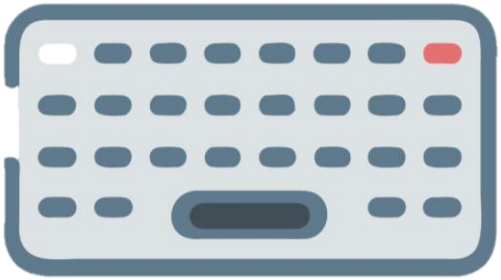
Virtual Keyboard in HCI

Dr. Salwa Osama



What is Virtual Keyboard?

Virtual keyboards are more than just digital typing tools — they are a key example of how humans interact with computers in modern systems.

They are used everywhere — in smartphones, tablets, ATMs, hospital systems, and even smart TVs — making them an essential part of Human–Computer Interaction

Why are Virtual Keyboards Important?

Accessibility:
They help people with **disabilities** type easily without needing a physical keyboard.

Portability:
You can use them on any touchscreen device — phone, tablet, or laptop — anywhere, anytime.

Security:
Virtual keyboards protect sensitive data (like passwords) from hackers and keyloggers.

Customization:
Users can change layout, color, and size to make typing more comfortable and personal.

Productivity:
Features like autocorrect and predictive text make typing faster and more efficient.

Health & Comfort:
Typing on-screen can reduce wrist strain (RSI) and be more natural for some users.

Virtual Keyboard Example

VKeyboard is the virtual keyboard shown on the screen

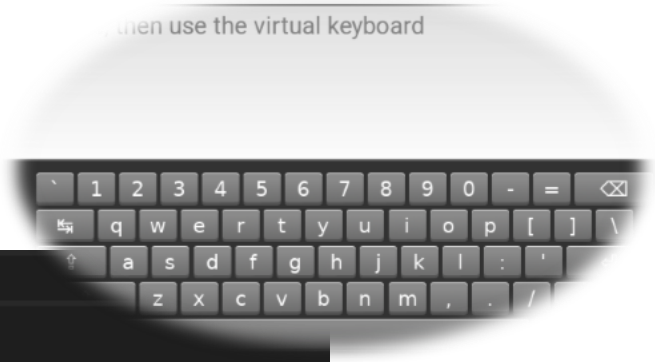
TextInput lets the user type text in a box

```
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.textinput import TextInput
from kivy.uix.vkeyboard import VKeyboard
```

BoxLayout helps to arrange items vertically or horizontally

This imports the App class which is used to create and run the application.

Virtual Keyboard Example



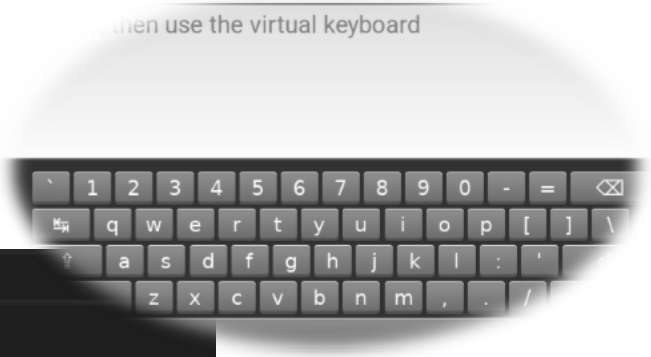

```
class KeyboardDemo(BoxLayout):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.orientation = "vertical"

        # Create text inpu
        self.text_input = extInput(
            multiline=Fal
            font_size=24,
            size_hint=(1,  2),
            hint_text="Cl  here, then use t  rtual keyboard"
        )
        self.add_widget(  text_input)

        # Create virtual  ard
        self.keyboard =  ard(on_key_up=self.key
        self.add_widget  eyboard)
```

This is the constructor. It starts the parent class BoxLayout

Makes the layout vertical (elements appear one below the other)

We create a new class called KeyboardDemo that uses BoxLayout to arrange elements.

Virtual Keyboard Example

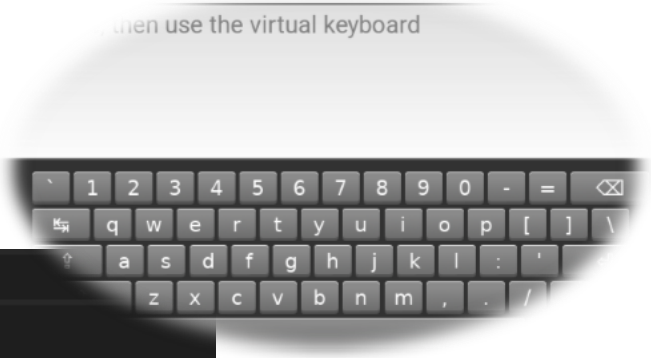



```python
class KeyboardDemo(BoxLayout):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.orientation = "vertical"

        # Create text input
        self.text_input = TextInput(
            multiline=False,
            font_size=24,
            size_hint=(1, 0.2),
            hint_text="Click here, then use th
        )
        self.add_widget(self.text_input)

        # Create virtual keyboard
        self.keyboard = VKeyboard(on_key_up=self.key_re
        self.add_widget(self.keyboard)
```

Adds the text box to the screen

multiline=False: only one line of text.
font_size=24: makes the text bigger.
size_hint=(1, 0.2): width = 100%, height = 20%.
hint_text: shows a small message inside the box before typing.

Virtual Keyboard Example




```python
class KeyboardDemo(BoxLayout):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.orientation = "vertical"

        # Create text input
        self.text_input = TextInput(
            multiline=False,
            font_size=24,
            size_hint=(1, 0.2),
            hint_text="Click here, then use the virtual keyboard"
        )
        self.add_widget(self.text_input)

        # Create virtual keyboard
        self.keyboard = VKeyboard(on_key_up=self.key_released)
        self.add_widget(self.keyboard)
```

Creates a virtual keyboard and calls the function key_released when a key is pressed.

Adds the keyboard to the screen under the text box

Virtual Keyboard Example

```
def key_released(self, keyboard, keycode, *args):
    key = keycode[1] if isinstance(keycode, tuple) else keycode
    print("Key pressed:", key)

    if key == 'backspace':
        self.text_input.text = self.text_input.text[:-1]
    elif key == 'spacebar':
        self.text_input.text += " "
    elif len(key) == 1:  # letter or number
        self.text_input.text += key
```

If keycode is a tuple (like (97, 'a')), we take the second element ('a').
Otherwise, we just use the value directly (like 'backspace').

Defines a function that runs when the user presses a key

Virtual Keyboard Example

If the key length is 1 (for example, 'a', 'b', or '5'), it means it's a normal letter or number, so we add it to the text.

```
def key_r                                  de, *args):
    key = keyc          if isinstance(keycode, tuple) else keycode
    print("Key          ed:", key)

    if key == 'b      pace':
        self.text      nput.text = self.text_input.text[:-1]
    el      key == '      acebar':
        elf.text_      nput.text +=      ' '
    eli      en(key) == 1:  # letter      r number
        f.text_input.text += key
```

If the pressed key is "backspace", remove the last character from the text box. [:-1] means "all characters except the last one".

If the key is "spacebar", add a space in the text input box.

Thanks